

Fun Optimizations For eBPF Programs and How To Enable Them

eBPF And Kernel Extension Workshop - 2026

Farbod Shahinfar, Aurojit Panda, Gianni Antichi



**POLITECNICO
MILANO 1863**



NEW YORK UNIVERSITY

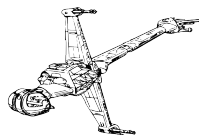


Queen Mary
University of London

Opening...

IN A SYSTEM NOT THAT OLD AND NOT THAT FAR AWAY ...

eBPF programs are not benefiting from the existing compiler optimizations applied to user space ones. This limits performance and exacerbates the trade-offs when offloading from user space to kernel. The underlying cause is a tension between eBPF verifier and compiler. Can we overcome the challenge by increasing flexibility? ...



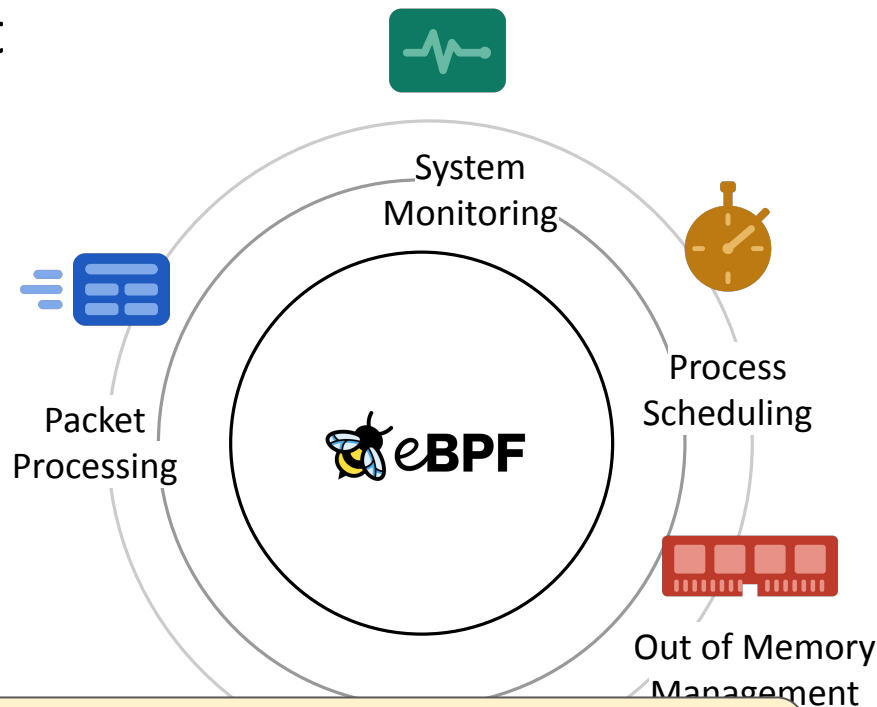
eBPF Programs Must Be Efficient

- Load balancer
- Intrusion Detection
- Task scheduler

...

They must operate
in μ s or less

Each CPU cycles
become even
more important



Performance is a requirement for these applications as they have limited processing budget to perform their task

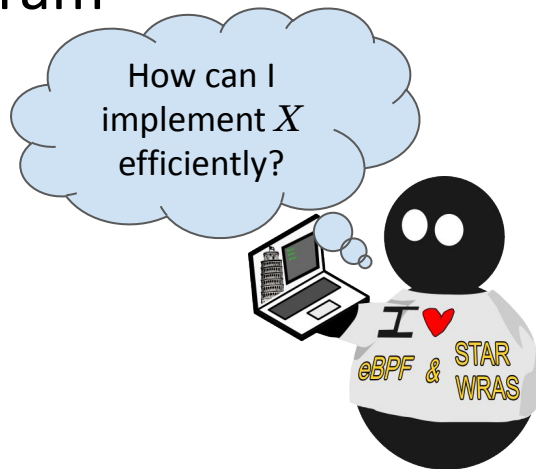
Optimization Opportunity For an eBPF Program

1. Efficient algorithms and data-structures

eNetSTL: Towards an In-kernel Library for High-Performance eBPF-based Network Functions

Bin Yang¹, Dian Shen^{1*}, Junxue Zhang^{2*}, Hanlin Yang¹, Lunqi Zhao¹, Beilun Wang¹,
Guyue Liu³, Kai Chen²

¹Southeast University ²Hong Kong University of Science and Technology ³Peking University



Fast In-kernel Traffic Sketching in eBPF

Sebastiano Miano
Queen Mary University of London
s.miano@qmul.ac.uk

Ran Ben Basat
University College London
r.benbasat@cs.ucl.ac.uk

Xiaoqi Chen
Princeton University
xiaoqi@cs.princeton.edu

Gianni Antichi
Politecnico di Milano and
Queen Mary University of London
gianni.antichi@polimi.it

A Memory Pool Allocator for eBPF Applications

Gyuyeong Kim
Sungshin Women's University
Seoul, Republic of Korea
gykim@sungshin.ac.kr

Dongsu Han
KAIST
Daejeon, Republic of Korea
dhan.ee@kaist.ac.kr

Optimization Opportunity For an eBPF Program

1. Efficient algorithms and data-structures
2. Runtime implementation
 - e.g., hook execution overhead, helper function costs, MAP operations, ...



Fast, Flexible, and Practical Kernel Extensions

Kumar Kartikeya Dwivedi[†] Rishabh Iyer^{*} Sanidhya Kashyap[†]
[†]EPFL ^{*}UC Berkeley



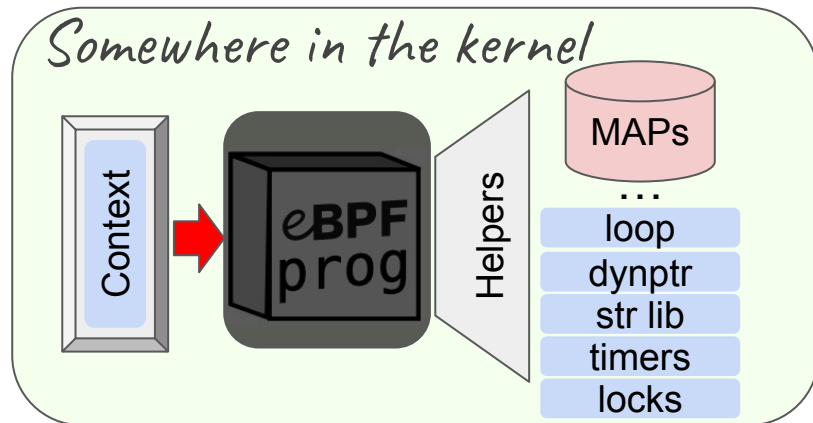
Don't Stall Me Now: Hiding Memory Latency in eBPF

Farbod Shahinfar
Politecnico di Milano
Italy, Milan

Marco Molè
Politecnico di Milano
Italy, Milan

Aurojit Panda
New York University
United States, New York

Gianni Antichi
Politecnico di Milano
Italy, Milan
Queen Mary University of London
United Kingdom, London

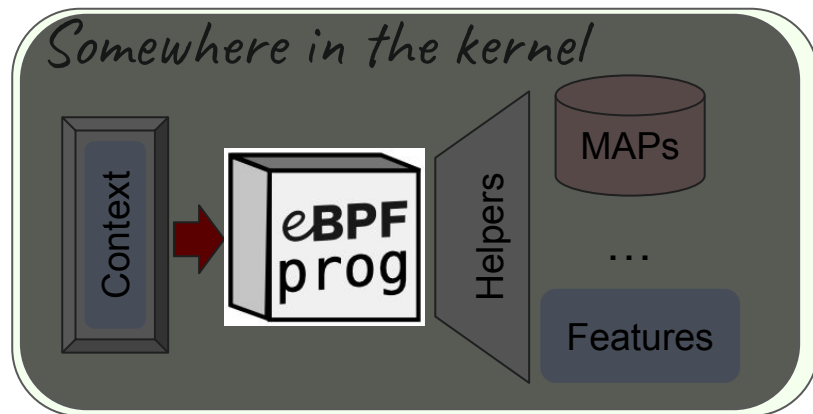


Optimization Opportunity For an eBPF Program

1. Efficient algorithms and data-structures
2. Runtime implementation
 - e.g., hook execution overhead, helper function costs, MAP operations, ...
3. Efficient implementation of program

High code quality

- Use of available CPU features
 - Instructions
 - Caches
 - Registers
 - Speculation pipeline
 - Memory Layout



Optimization Opportunity For an eBPF Program

1. Efficient algorithms and data-structures
2. Runtime implementation
 - e.g., hook execution overhead, helper function costs, MAP operations, ...

3. Efficient implementation of program

- High code quality
- Use of available CPU features
 - Instructions
 - Caches
 - Registers
 - Speculation pipeline
 - Memory Layout



This is what compilers try to optimize and is the focus of this presentation

Optimization Opportunity For an eBPF Program

1. Efficient algorithms and data-structures
2. Runtime implementation
 - e.g., hook execution overhead, helper function costs, MAP operations, ...
3. Efficient implementation of program
 - Use of available CPU features
 - Instructions
 - Caches
 - Registers

code quality

MERLIN: Multi-tier Optimization of eBPF Code for Performance and Compactness

Jinsong Mao
jinsongmao@umass.edu
University of Massachusetts Amherst
USA

Juan Zhai
juanzhai@umass.edu
University of Massachusetts Amherst
USA

Hailun Ding
hailun.ding@rutgers.edu
Rutgers University
USA

Shiqing Ma
shiqingma@umass.edu
University of Massachusetts Amherst
USA

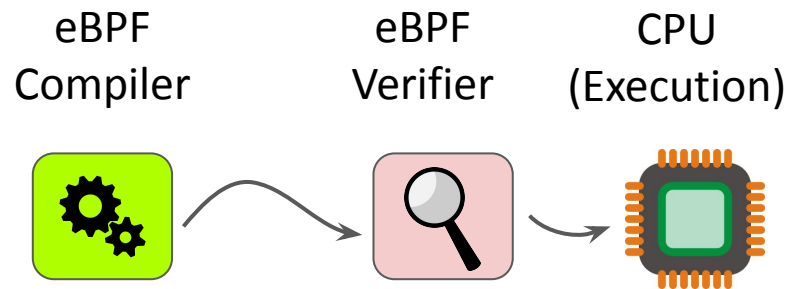
Synthesizing Safe and Efficient Kernel Extensions for Packet Processing

Qiongwen Xu¹, Michael D. Wong², Tanvi Wagle¹, Srinivas Narayana¹, Anirudh Sivaraman³
¹Rutgers University ²Princeton University ³New York University
k2_compiler@email.rutgers.edu



Tension Between eBPF Compiler and Verifier

Only verified eBPF programs run in the kernel
This creates tension:

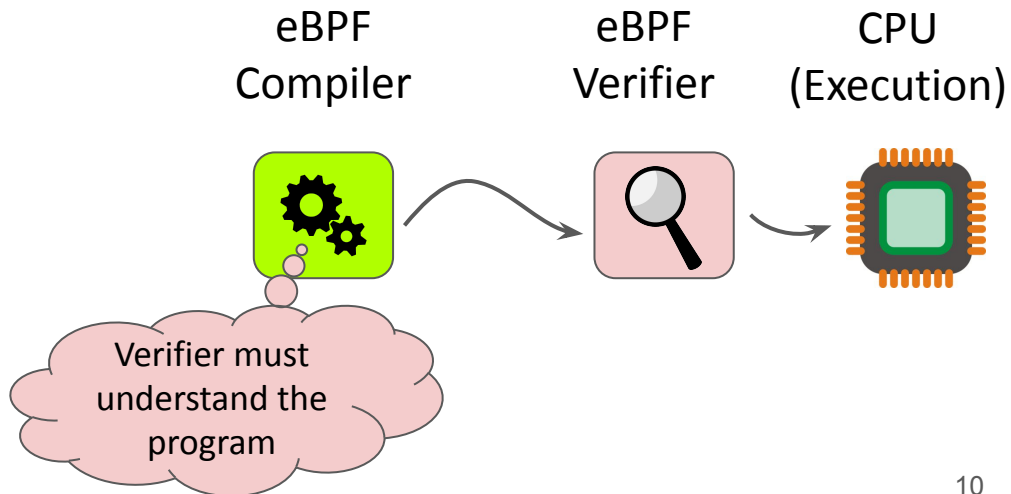


Tension Between eBPF Compiler and Verifier

Only verified eBPF programs run in the kernel

This creates tension:

- Verifier needs to understand the program

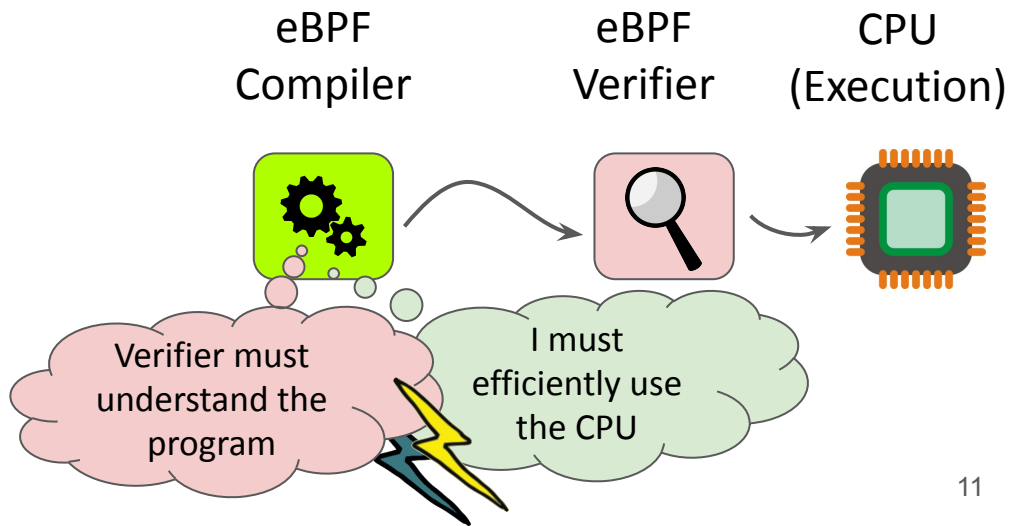


Tension Between eBPF Compiler and Verifier

Only verified eBPF programs run in the kernel

This creates tension:

- Verifier needs to understand the program
- Compiler wants to optimize the program which can remove information



A Concrete Example

1. Figure shows a program that reads 20 bytes of receiving packets and performs some computation

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

A Concrete Example

1. Figure shows a program that reads 20 bytes of receiving packets and performs some computation
2. The program does not pass the verifier despite:
 - Using a bounded loop with constant upper bound of 20 ← [Information for program termination](#)
 - Performing bound check for every byte of packet accessed ← [Information for memory safety](#)

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

A Concrete Example

Checking the generated program with Clang

Compiler hoists bound check and uses packet size as an upper bound (L6-8)

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

```
1 r4 = r3                ; r4 = data_end
2 r4 += 0x1              ; r4 = data_end + 1
3 r1 = r2                ; r1 = data
4 r1 += 0x1              ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1                ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2                ; <--
11 ; ...                  ; |
12 if r4 != 0x14 goto -0x7 ; --+
```

A Concrete Example

Checking the generated program with Clang

Compiler hoists bound check and uses packet size as an upper bound (L6-8):

- (1): computes pkt size

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

```
1 r4 = r3                ; r4 = data_end
2 r4 += 0x1              ; r4 = data_end + 1
3 r1 = r2                ; r1 = data
4 r1 += 0x1              ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1                ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2                ; <--
11 ; ...                  ; |
12 if r4 != 0x14 goto -0x7 ; --+
```

1

A Concrete Example

Checking the generated program with Clang

Compiler hoists bound check and uses packet size as an upper bound (L6-8):

- (1): computes pkt size
- (2): checks pkt to bigger than 20 B

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

```
1 r4 = r3                ; r4 = data_end
2 r4 += 0x1              ; r4 = data_end + 1
3 r1 = r2                ; r1 = data
4 r1 += 0x1              ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1                ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2                ; <--
11 ; ...                  ; |
12 if r4 != 0x14 goto -0x7 ; --+
```

1

2

A Concrete Example

Checking the generated program with Clang

Compiler hoists bound check and uses packet size as an upper bound (L6-8):

- (1): computes pkt size
- (2): checks pkt to bigger than 20 B
- No more check in the body of the loop

```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

```
1 r4 = r3                ; r4 = data_end
2 r4 += 0x1              ; r4 = data_end + 1
3 r1 = r2                ; r1 = data
4 r1 += 0x1              ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1                ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2                ; <--
11 ; ...                  ; |
12 if r4 != 0x14 goto -0x7 ; --+
```

1

2

A Concrete Example

Checking the generated program with Clang

Compiler hoists bound check and uses packet size as an upper bound (L6-8):

- (1): computes pkt size
- (2): checks pkt to bigger than 20 B
- No more check in the body of the loop

The verifier can not reason about memory bounds

The verifier error message: invalid access to packet, off=0 size=1, R5(id=0,off=0,r=0); R5 offset is outside of the packet

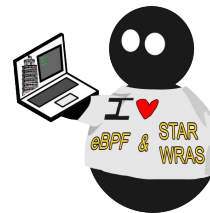
```
1 #pragma clang loop unroll(disabled)
2 for (__u16 i = 0; i < 20; i++) {
3     if ((void *)(p + i + 1) > data_end)
4         return XDP_DROP;
5     unsigned char x = p[i];
6     c += x;
7     c <= 8;
8 }
```

```
1 r4 = r3 ; r4 = data_end
2 r4 += 0x1 ; r4 = data_end + 1
3 r1 = r2 ; r1 = data
4 r1 += 0x1 ; r1 = data + 1
5 ; ...
6 r4 = r2
7 r4 -= r1 ; r4 = -(pkt size)
8 if r4 > -0x15 goto +0x20 ; --> exit
9 r4 = 0x0
10 r5 = r2 ; <--
11 ; ... ; |
12 if r4 != 0x14 goto -0x7 ; -->
```

1

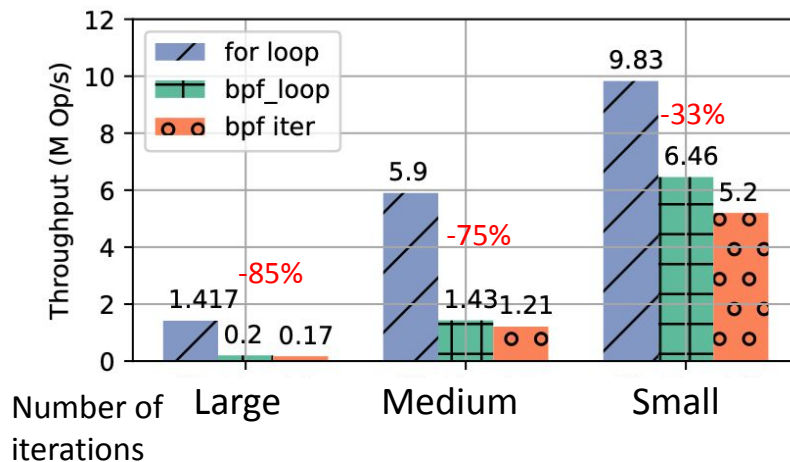
2

The Tension Affects Developers

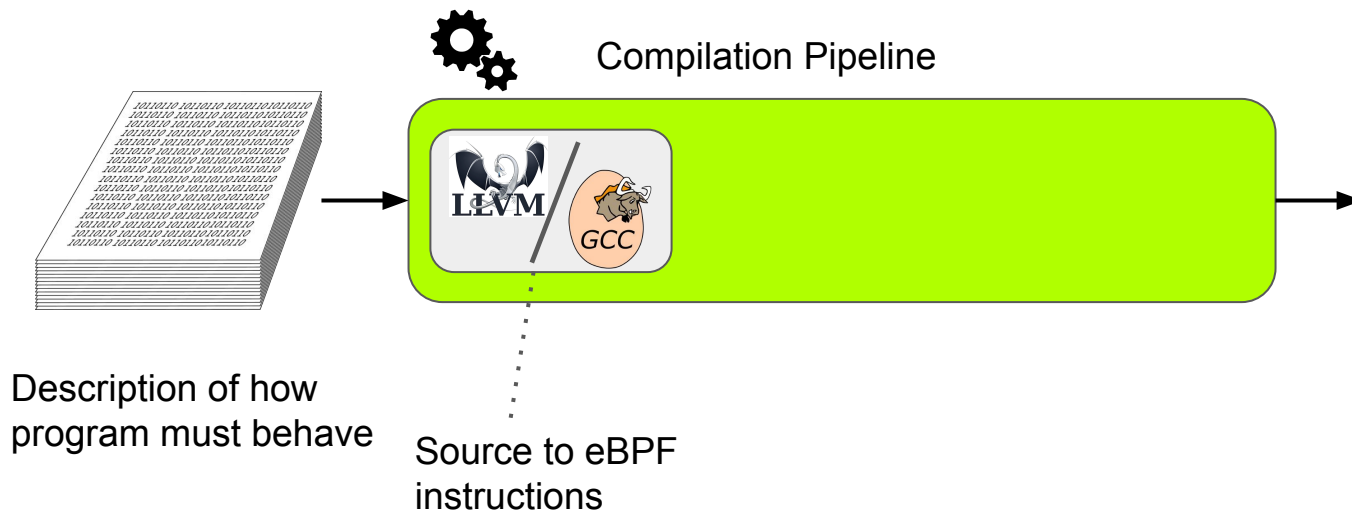


- The verification restrictions also forces developers to sacrifice performance.
- For example comparing C-for-loops with bpf_loop, and BPF Iter shows 33 to 85% of overhead

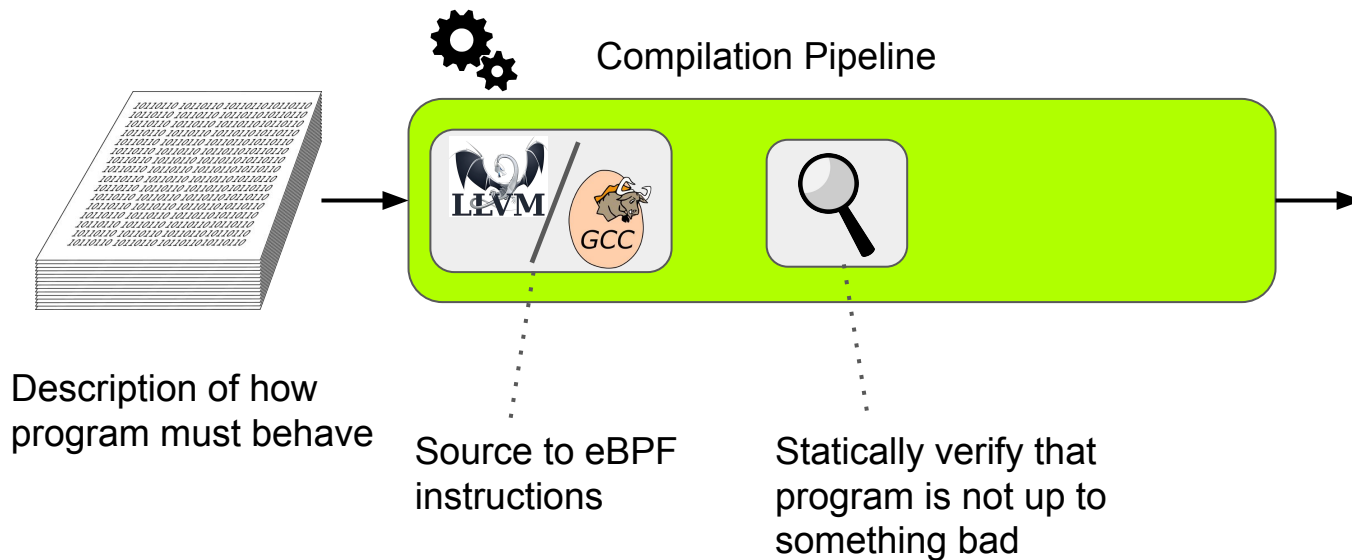
Benchmark:
Computing Fibonacci
number



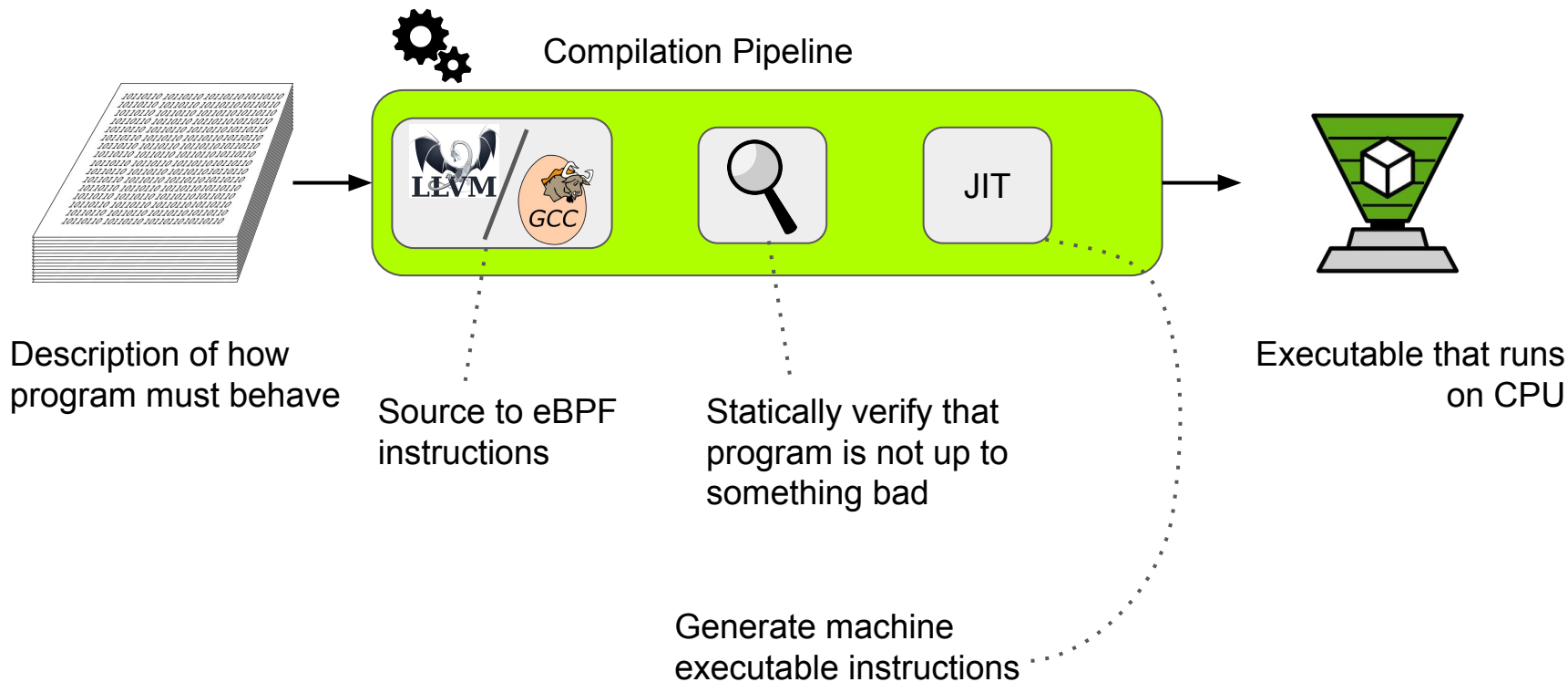
eBPF's Compiler Pipeline Is Special



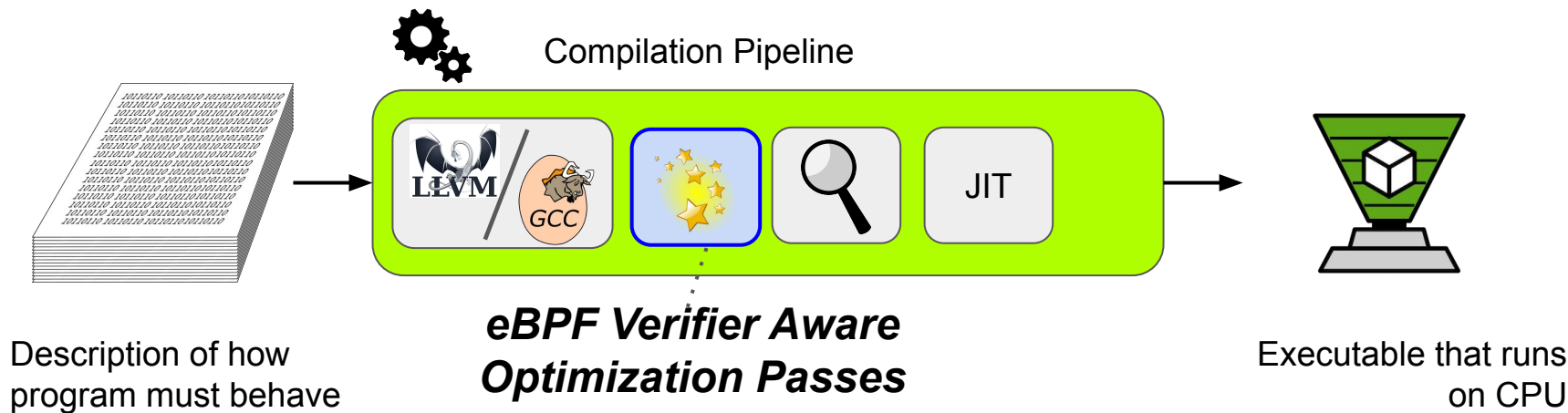
eBPF's Compiler Pipeline Is Special



eBPF's Compiler Pipeline Is Special



eBPF's Compiler Pipeline Is Special



MERLIN: Multi-tier Optimization of eBPF Code for Performance and Compactness

Jinsong Mao
jinsongmao@umass.edu
University of Massachusetts Amherst
USA

Juan Zhai
juanzhai@umass.edu
University of Massachusetts Amherst
USA

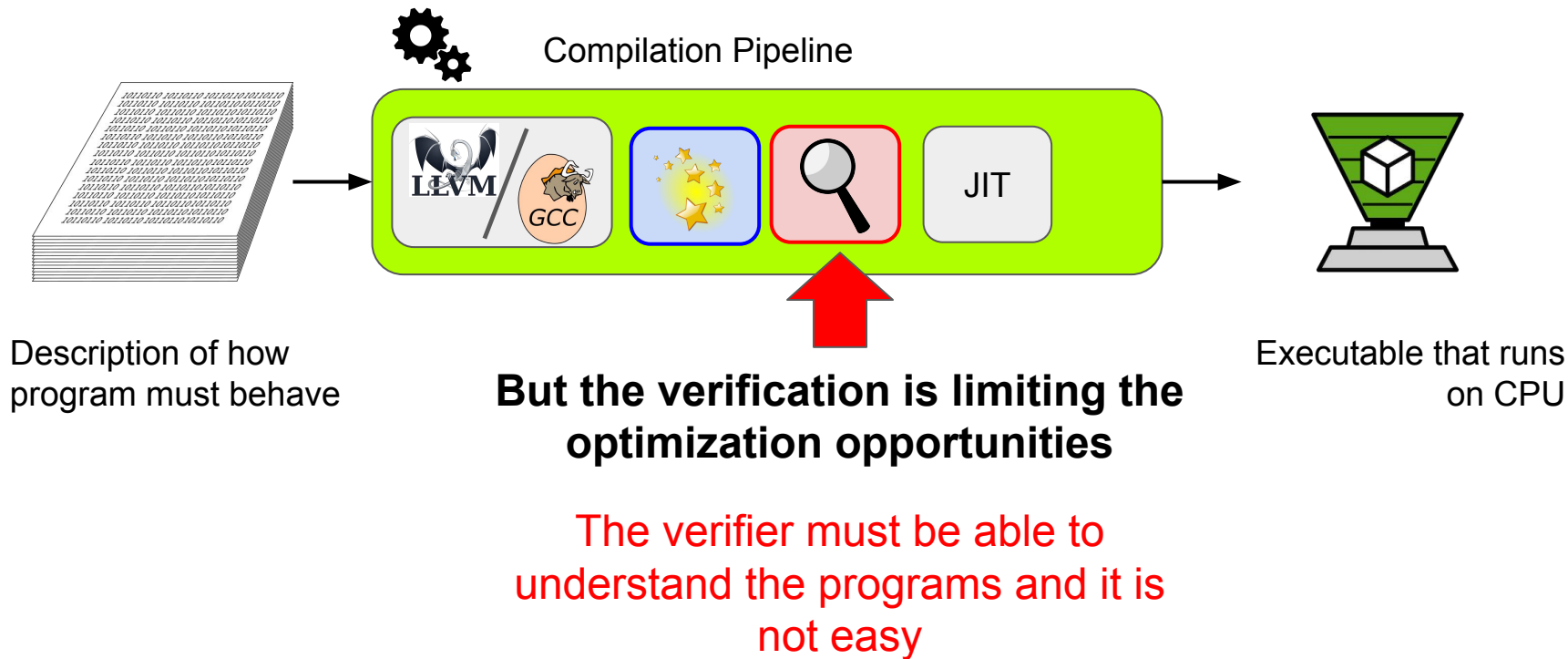
Hailun Ding
hailun.ding@rutgers.edu
Rutgers University
USA

Shiqing Ma
shiqingma@umass.edu
University of Massachusetts Amherst
USA

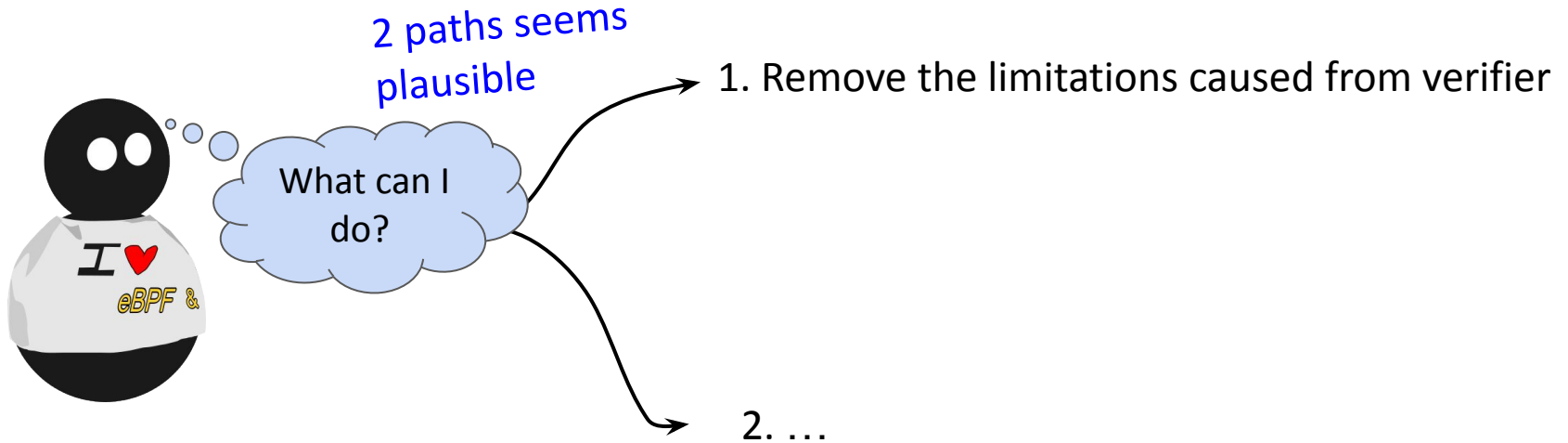
Synthesizing Safe and Efficient Kernel Extensions for Packet Processing

Qiongwen Xu¹, Michael D. Wong², Tanvi Wagle¹, Srinivas Narayana¹, Anirudh Sivaraman³
¹Rutgers University ²Princeton University ³New York University
k2_compiler@email.rutgers.edu

eBPF's Compiler Pipeline Is Special



There seems to be two possible paths forward...



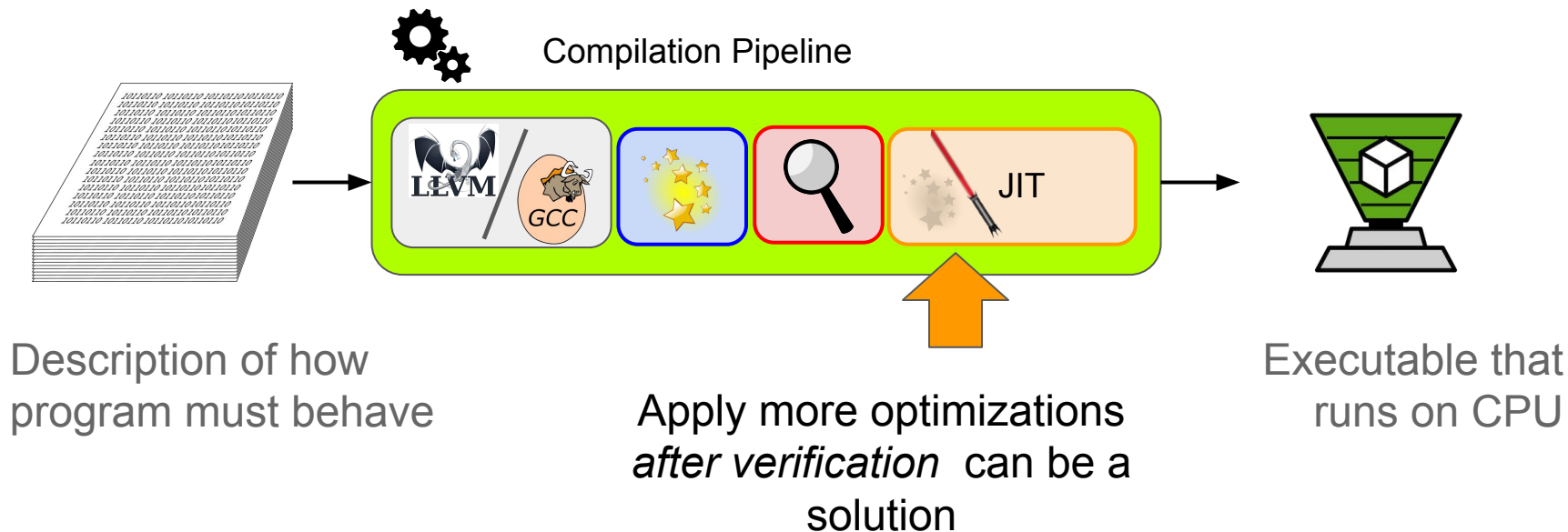
Verifier Is Important

- Verifier is a core component of eBPF and it is very valuable:
 - Kernel extension without the risk of breaking the kernel
- Removing it is not an option!

Verifier Is Important

- Verifier is a core component of eBPF and it is very valuable:
 - Kernel extension without the risk of breaking the kernel
- Removing it is not an option!
- Improving the verifier to accept more optimizations
 - Fixed computation and memory budget
 - There will always be properties that verifier cannot reason about

Proposal



What it enables?

- Replacing the `bpf_loop` and fully inlining the logic
 - The kernel already does one level of inlining
- Improving register allocations
- Using advanced instructions: e.g, `rep movs`, and more
- **Profile guided optimization**

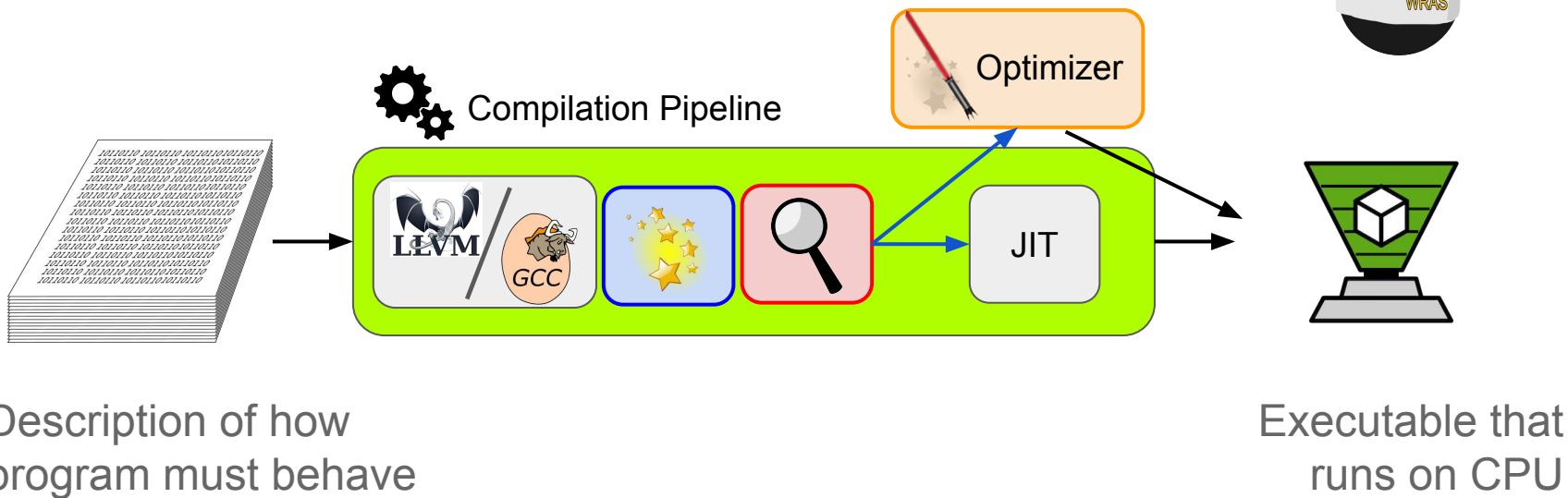
What it enables?

- Replacing the `bpf_loop` and fully inlining the logic
 - The kernel already does one level of inlining
- Improving register allocations
- Using advanced instructions: e.g, `rep movs`, and more
- **Profile guided optimization**

An incorrect optimizer can introduce bugs into the verified program and nullify the verifier's work

Our Suggestion

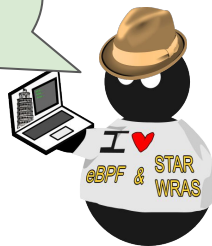
- Apply optimization after verification in user space!



Our Suggestion

- Apply optimization after verification in user space!

I am a super user and I want that optimization



Description of how

Executable that

Relying on a privileged user for introducing the optimizer moves the responsibility of safe program behavior from kernel to the user

A Question

- Our suggested approach shifts focus from verifying complex programs to verifying the optimizations. Is it an acceptable direction?

Thank you very much

Contact: farbod.shahinfar@polimi.it

